EdgeCount: Statistical Theory and Algorithmic Efficiency

Joel R. Pradines

December 8, 2025

Abstract

This vignette details the statistical framework underlying the `EdgeCount` package. First, the Random Graph with Given Expected Degrees (RGGED) model is described. Second, the derivation of edge-count statistics is presented. Third, the algebraic optimizations that allow the package to calculate connectedness statistics with linear time complexity are explained. Last, the implementation of vectorized functions contained in the package is discussed, and it is shown that they have optimal time complexity.

1 Introduction

The `EdgeCount` package provides methods to analyze the connectedness of vertex sets in large, sparse, undirected graphs. Whether analyzing interaction networks or bipartite group-element graphs, the question is:

Is an observed number of edges, called edge count, within a set (or between two sets) significantly higher than expected by chance?

To answer this, two components must be implemented.

First, the random experiment defining the concept of *chance* must be clearly specified and, in general, simulated to provide a control distribution.

Second, one needs to be able to fairly compare edge-count values to each other. A fair comparison requires accounting for vertex degrees, as these may vary by orders of magnitude. This is achieved here by using a null model of random graphs that preserves the expected degrees of vertices [3, 4]. Specifically, the average degree of a vertex over all random graph realizations matches its observed value in the original network. The analytical expressions for the likelihoods of edge-count variables in this model were derived in [13]. These expressions enable fair and fast comparisons of edge-count values.

2 Random graph model

Let $G(V, E)$ be an undirected graph with no loops and no multiple edges.

- Let $N = |V|$ be the order of the graph (number of vertices).
- Let $M = |E|$ be the size of the graph (number of edges).
- Let k_i be the degree of vertex v_i.
- Let $\langle k \rangle$ be the the average vertex degree.

Call degree sequence of G the nonincreasing sequence of all vertex degrees. An obvious null model for G is that of a random graph whose realizations always preserve the degree sequence. This model is referred to as the Random Graph with Fixed Degree Sequence (RGFDS). However, it is known that the analytical treatment of the RGFDS is a difficult problem [2, 11, 5, 12]. Namely, deriving simple analytical expressions for the likelihoods of edge-count variables is in general hard with the RGFDS. This implies that slow numerical methods based on simulations must be used [8, 10, 9].

A less constrained random graph model than the RGFDS is the Random Graph with Given Expected Degrees (RGGED) [3, 4]. While an individual realization of the RGGED might not preserve a vertex degree, it is preserved on average over all realizations. The key feature of the RGGED is that edges are treated as independent random variables, enabling the derivation of analytical expressions for the distributions of edge-count variables [13].

In the RGGED model, edges are treated as independent Bernoulli random variables: an edge $v_i v_j$ exists with probability p_{ij} (Bernoulli parameter) and does not exist with probability $1 - p_{ij}$. When the graph is large and sparse on average over its vertices ($\langle k \rangle^2 \ll 2M$), the probability p_{ij} of observing an edge between vertices v_i and v_j is given by [13]:

$$p_{ij} \simeq 1 - \exp\left(-k_i k_j / (2M)\right). \tag{1}$$

Useful upper bounds for p_{ij} are

$$p_{ij} \leq \min\left(1, \frac{k_i k_j}{2M}\right) \leq \frac{k_i k_j}{2M}. \tag{2}$$

2.1 Edge-count distributions

Edge-count random variables in the RGGED model of a large and sparse graph have approximate Poisson distribution. This is because they are the sum of independent Bernoulli random variables with small parameters $p_{ij} \ll 1$ [7, 6, 13].

Call Z an edge-count random variable defined by the sum of m potential edges, each edge corresponding to a pair $v_i v_j$ of vertices:

$$Z = \sum_{u=1}^{m} B_u \quad \text{with} \quad B_u \sim \text{Bernoulli}\left(p_u = p_{ij}\right). \tag{3}$$

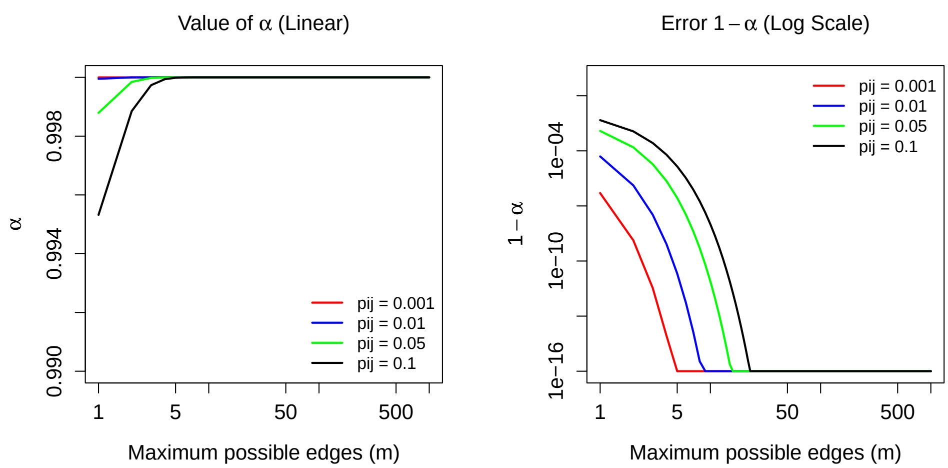
Then we have $Z \sim \text{Poisson}\left(\lambda, m\right)$ as defined by

$$\Pr\left(Z = z\right) = \frac{e^{-\lambda}}{\alpha} \frac{\lambda^z}{z!} \quad \text{with} \quad \lambda = \sum_{u=1}^{m} p_u \quad \text{and} \quad \alpha = e^{-\lambda} \sum_{x=0}^{m} \frac{\lambda^x}{x!}. \tag{4}$$

The normalization factor α is utilized because an observed number z of edges is bounded by the maximum number m of edges. This said, we have $\alpha \simeq 1$ when $p_{ij} \ll 1$.

```
> # Function to calculate alpha and its complement
> get_alpha_stats <- function(m, pij_avg) {
+   lambda <- m * pij_avg
+   alpha <- ppois(m, lambda)
+   alpha_comp <- ppois(m, lambda, lower.tail = FALSE)
+   return(c(alpha, alpha_comp))
+ }
> # Parameters
> pij_values <- c(0.001, 0.01, 0.05, 0.1)
> m_values <- c(1:29,seq(30, 1000, by = 10))
> colors <- c("red", "blue", "green", "black")
> # Plots
> par(mfrow=c(1, 2), mar=c(5, 5, 4, 2) + 0.1, cex.lab=1.25,
+     cex.axis=1.25, cex.main=1.25)
> plot(1, type="n", xlab="Maximum possible edges (m)",
+      ylab=expression(alpha),
+      xlim=c(1, 1000), ylim=c(0.99, 1.00),
+      log="x",
+      main=expression(paste("Value of ", alpha, " (Linear)")))
> for(i in seq_along(pij_values)) {
```

```
+   lambdas <- m_values * pij_values[i]
+   y_vals <- ppois(m_values, lambdas)
+   lines(m_values, y_vals, col=colors[i], lwd=2)
+ }
> legend("bottomright", legend=paste("pij =", pij_values),
+        col=colors, lwd=2, bty="n", cex=1)
> epsilon <- 1e-16
> plot(1, type="n", xlab="Maximum possible edges (m)",
+      ylab=expression(1 - alpha),
+      xlim=c(1, 1000), ylim=c(epsilon, 1),
+      log="xy",
+      main=expression(paste("Error ", 1 - alpha, " (Log Scale)")))
> for(i in seq_along(pij_values)) {
+   lambdas <- m_values * pij_values[i]
+   y_vals <- ppois(m_values, lambdas, lower.tail = FALSE)
+   y_vals <- pmax(y_vals, epsilon)
+   lines(m_values, y_vals, col=colors[i], lwd=2)
+ }
> legend("topright", legend=paste("pij =", pij_values),
+        col=colors, lwd=2, bty="n", cex=1)
```



2.2 Edge-count p-values

2.3 Edge-count effect sizes

3 Fast approximations for sparse graphs

For sparse graphs where $\max(k_i k_j) \ll 2M$, this approximates the probability of edge formation in random graphs with fixed degrees. When the graph is sufficiently sparse (as determined by the package function `summarize_suitability_fast`), we can use the approximation:

$$p_{ij} \approx \frac{k_i k_j}{2M} \tag{5}$$

This approximation is central to the algorithmic efficiency of the package, as it allows for the factorization of degree sums.

4 Edge Count Probabilities

Let Z be a random variable representing the count of edges in a specific subgraph configuration (e.g., edges within a set). Z is the sum of independent Bernoulli trials. While the exact distribution of Z is a Poisson-Binomial distribution, it is well-approximated by a Poisson distribution (or a truncated Poisson distribution) when probabilities p_{ij} are small [13].

The probability of observing z edges is approximated by:

$$P(Z = z) = \frac{e^{-\lambda}}{\alpha} \frac{\lambda^z}{z!} \tag{6}$$

where:

- $\lambda = \mathbb{E}[Z] = \sum p_{ij}$ is the expected number of edges.
- $\alpha = e^{-\lambda} \sum_{h=0}^{m} \frac{\lambda^h}{h!}$ is the normalization factor accounting for the maximum possible number of edges m in the subgraph.

The p-value, or the likelihood of observing x or more edges, is:

$$\text{p-value} = P(Z \geq x) = \frac{e^{-\lambda}}{\alpha} \sum_{h=x}^{m} \frac{\lambda^h}{h!} \tag{7}$$

The addition of $3/8$ stems from the work of [1], who showed that $\sqrt{X + 3/8}$ is the optimal variance-stabilizing transformation for Poisson data. The lAr metric is effectively the log-ratio of these variance-stabilized values.

5 Algorithmic Optimization

A key feature of `EdgeCount` is its speed. Calculating λ involves summing probabilities over pairs of vertices. A naive implementation requires iterating over all pairs, resulting in quadratic time complexity. However, by utilizing the "Fast" approximation (Equation 5), we can factorize these sums to achieve linear time complexity.

5.1 Within-Set Connectedness

Consider a set of vertices $S \subseteq V$ with size $n = |S|$. We wish to calculate the expected number of edges λ_{in} existing internally among members of S.

$$\lambda_{in} = \sum_{\{i,j\} \subseteq S, i<j} p_{ij} \approx \sum_{i<j} \frac{k_i k_j}{2M} \tag{8}$$

A naive summation requires $O(n^2)$ operations. To optimize this, we recall the algebraic identity for the square of a sum:

$$\left(\sum_{i \in S} k_i \right)^2 = \sum_{i \in S} k_i^2 + 2 \sum_{i<j} k_i k_j \tag{9}$$

Rearranging for the cross-terms:

$$\sum_{i<j} k_i k_j = \frac{1}{2} \left[\left(\sum_{i \in S} k_i \right)^2 - \sum_{i \in S} k_i^2 \right] \tag{10}$$

Substituting this into the expression for λ_{in}:

$$\lambda_{in} \approx \frac{1}{2M} \cdot \frac{1}{2} \left[\left(\sum_{i \in S} k_i \right)^2 - \sum_{i \in S} k_i^2 \right] = \frac{1}{4M} \left[\left(\sum_{i \in S} k_i \right)^2 - \sum_{i \in S} k_i^2 \right] \tag{11}$$

Complexity: We now only need to compute $\sum k_i$ and $\sum k_i^2$. This requires iterating through S once, reducing the complexity to $O(n)$. This optimization is implemented in the package function `calculate_in_stats_fast_vectorized`.

5.2 Between-Set Connectedness

Consider two disjoint sets of vertices S_1 and S_2. We wish to calculate the expected number of edges λ_{bt} bridging these two sets.

$$\lambda_{bt} = \sum_{i \in S_1} \sum_{j \in S_2} p_{ij} \approx \sum_{i \in S_1} \sum_{j \in S_2} \frac{k_i k_j}{2M} \tag{12}$$

A naive summation requires $O(|S_1| \cdot |S_2|)$ operations. Because the terms k_i and k_j are independent in the product, we can factor the sum:

$$\lambda_{bt} \approx \frac{1}{2M} \left(\sum_{i \in S_1} k_i \right) \left(\sum_{j \in S_2} k_j \right) \tag{13}$$

Complexity: We sum the degrees in S_1 ($O(|S_1|)$) and S_2 ($O(|S_2|)$) separately and multiply the results. The total complexity is $O(|S_1| + |S_2|)$, which is linear relative to the input size. This optimization is implemented in `calculate_between_stats_fast_vectorized`.

6 Effect Size: The Anscombe Ratio

While the p-value indicates significance, it is dependent on sample size. To measure the magnitude of connectedness, `EdgeCount` employs the log Anscombe Ratio (lAr), which stabilizes the variance of Poisson-distributed variables.

$$lAr = \frac{1}{2} \log_2 \left(\frac{x + 3/8}{\lambda + 3/8} \right) \tag{14}$$

where x is the observed edge count and λ is the expected edge count.

- $lAr > 0$: The set is more connected than expected (enrichment).
- $lAr < 0$: The set is less connected than expected (depletion).

The addition of 3/8 is an optimal correction for bias in the log transformation of Poisson variables [14].

7 Vertex Set Enrichment Analysis (VSEA)

The VSEA method implemented in `run_vsea` adapts the principles of Gene Set Enrichment Analysis [15] to graph topology.

Standard GSEA uses a Kolmogorov-Smirnov-like running sum statistic. In VSEA, the "local" correlation at rank i is replaced by the edge-count probability framework derived by [13]. As we traverse the ranked list of elements, at step i (considering the top i elements as set S_i), we calculate:

1. The observed edges x_i between S_i and a target Term T.
2. The expected edges λ_i using the RGGED model.
3. The running score based on the Log Anscombe Ratio.

Crucially, because λ_i accounts for the degrees of the elements in S_i, **VSEA inherently penalizes hubs**. If the top-ranked elements are merely "sticky" (high degree) but not specifically connected to Term T, λ_i will be very high, reducing the lAr score. This distinguishes VSEA from standard hypergeometric enrichment which only counts overlaps, ignoring the varying likelihood of connections for high-degree elements.

References

[1] Francis J. Anscombe. The transformation of poisson, binomial and negative-binomial data. *Biometrika*, 35(3/4):246–254, 1948.

[2] Edward A Bender and E Rodney Canfield. The asymptotic number of labelled graphs with given degree sequences. *Journal of Combinatorial Theory, Series A*, 24(3):296–307, 1978.

[3] Fan Chung and Linyuan Lu. Connected components in random graphs with given expected degree sequences. *Annals of Combinatorics*, 6(2):125–145, 2002.

[4] Fan R. K. Chung and Linyuan Lu. The average distances in random graphs with given expected degrees. *Proceedings of the National Academy of Sciences of the United States of America*, 99:15879–15882, 2002.

[5] S Itzkovitz, R Milo, N Kashtan, G Ziv, and U Alon. Subgraphs in random networks. *Physical Review E*, 68(2):026127, 2003.

[6] Johannes Kerstan. Verallgemeinerung eines satzes von prochorow und le cam. *Zeitschrift für Wahrscheinlichkeitstheorie und verwandte Gebiete*, 2(3):173–179, 1964.

[7] Lucien Le Cam. An approximation theorem for the poisson binomial distribution. *Pacific Journal of Mathematics*, 10(4):1181–1197, 1960.

[8] Sergei Maslov and Kim Sneppen. Specificity and stability in topology of protein networks. *Science*, 296(5569):910–913, 2002.

[9] R Milo, N Kashtan, S Itzkovitz, MEJ Newman, and U Alon. Uniform generation of random graphs with arbitrary degree sequences. *arXiv preprint cond-mat/0312028*, 2003.

[10] Ron Milo, Shai Shen-Orr, Shalev Itzkovitz, Nadav Kashtan, Dmitri Chklovskii, and Uri Alon. Network motifs: simple building blocks of complex networks. *Science*, 298(5594):824–827, 2002.

[11] Michael Molloy and Bruce Reed. A critical point for random graphs with a given degree sequence. *Random Structures & Algorithms*, 6(2-3):161–179, 1995.

[12] Juyong Park and M. E. J. Newman. Statistical mechanics of networks. *Physical Review E*, 70(6):066117, 2004.

[13] J Pradines, V Farutin, S Rowley, and V Dancik. Analyzing protein lists with large networks: edge-count probabilities in random graphs with given expected degrees. *Journal of Computational Biology*, 12(2):113–128, 2005.

[14] JR Pradines, N Cilfone, A Ghavami, V Farutin, E Kurtagic, J Guess, A Manning, and I Capila. Enhancing reproducibility of gene-expression analysis with known protein functional relationships: the concept of well-associated protein. *PLOS Computational Biology*, 16(2):e1007684, 2020.

[15] A Subramanian, P Tamayo, VK Mootha, S Mukherjee, BL Ebert, MA Gillette, A Paulovich, SL Pomeroy, TR Golub, ES Lander, and JP Mesirov. Gene set enrichment analysis: a knowledge-based approach for interpreting genome-wide expression profiles. *Proceedings of the National Academy of Sciences*, 102(43):15545–15550, 2005.